In this work, a decision tree classifier was implemented from scratch using Gini impurity for split evaluation and it does the classification of data into different type. Here both deterministic (optimal) and randomized splitting strategies are implemented. Based on this custom decision tree, custom Random Forest and custom Extra Trees ensemble methods were also implemented and compared with sklearn model.

**Decision tree structure**：Each node in the decision tree stores the Gini impurity, the number of samples reaching the node, class distribution, and split information consisting of a feature index and threshold.

**Stopping conditions (pre-pruning)**

Tree growth is stopped when one of the following conditions is met:

- The maximum tree depth is reached
- All samples at a node belong to the same class (Gini impurity equals zero)
- The number of samples is insufficient to perform a split

**Random Forest Implementation:**

For creating each tree, a subset of training data and subset of features are used. As subset of data is used, number of minimum samples per split is configurable. The feature subsampling reduces correlation among trees and improves generalization. Along with that, total number of trees in the forest is also configurable. The decision trees created here use best split for finding best feature to split the data. A number of decision tree is created based on splitted training data and splitted features and then majority voting is done for getting the final classification.

**Extra Tree Implementation:**

For creating each tree, whole training data and subset of features are used. But unlike random forest, it split data on random value of an attribute. All others are same as random forest

**Prediction and Probability Estimation:**

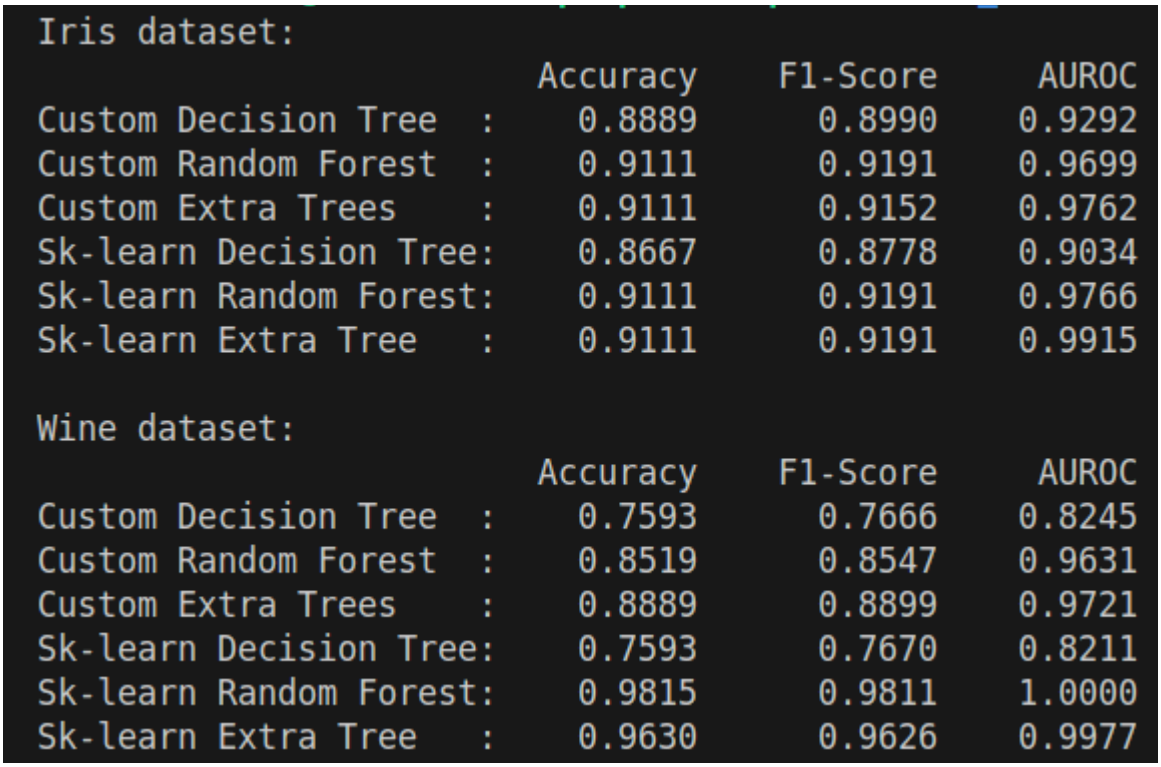
For probability-based evaluation metrics such as AUROC, class probabilities are estimated at leaf nodes using the normalized class distribution. Ensemble probabilities are computed by averaging probabilities across trees.

**Experimental Setup:**

```
test_data_size = 0.3
iris_max_depth = 2
wine_max_depth = 6
num_trees = 5
min_sample_per_split = 10
```

Test and train data is splitted in 3:7 ratio. Maximum tree depth is different for two dataset as iris dataset has only 4 features and more depth overfit the data. Number of trees in the forest is set as 5 here as both the dataset are relatively small and lower number of trees can give a nearly optimised result.

**Result and Analysis:**



```
Iris dataset:
                          Accuracy    F1-Score     AUROC
Custom Decision Tree  :     0.8889      0.8990    0.9292
Custom Random Forest  :     0.9111      0.9191    0.9699
Custom Extra Trees    :     0.9111      0.9152    0.9762
Sk-learn Decision Tree:     0.8667      0.8778    0.9034
Sk-learn Random Forest:     0.9111      0.9191    0.9766
Sk-learn Extra Tree   :     0.9111      0.9191    0.9915

Wine dataset:
                          Accuracy    F1-Score     AUROC
Custom Decision Tree  :     0.7593      0.7666    0.8245
Custom Random Forest  :     0.8519      0.8547    0.9631
Custom Extra Trees    :     0.8889      0.8899    0.9721
Sk-learn Decision Tree:     0.7593      0.7670    0.8211
Sk-learn Random Forest:     0.9815      0.9811    1.0000
Sk-learn Extra Tree   :     0.9630      0.9626    0.9977
```

Random Forest gives more accurate result as it uses more tree to find the class of the data where decision tree uses only one tree. Extra tree split data based on random value and so it may be better or may be worse in different case. But Extra tree need less computational cost than Random Forest. Finally, all the results are compared with scikit-learn implementation. Here, iris dataset gives optimal result as it is a small dataset with only 4 features and 150 data. As wine dataset is larger, it deviates slightly from the optimized result.